

Software Requirements Specification (SRS)

BroadcastSim

A Broadcast Network Simulation Platform

Version: 1.0

Status: Final

Technology Stack: Spring Boot • Thymeleaf • PostgreSQL • WebSocket

1. Introduction

1.1 Purpose

BroadcastSim is an interactive broadcast network simulation platform that enables users to design, configure, and simulate a television broadcasting infrastructure without requiring physical hardware.

The simulator allows users to:

- Build broadcast topologies
- Configure broadcast devices
- Modify engineering parameters
- Inject failures
- Observe cascading effects throughout the network
- Monitor device health in real time

The objective is to provide an educational and engineering-focused environment similar to Cisco Packet Tracer, but specifically for broadcast systems.

1.2 Scope

BroadcastSim is a logical simulator.

It does not process actual video streams.

Instead, it simulates:

- Broadcast devices

- Signal flow
 - Device behavior
 - Engineering parameters
 - Fault propagation
 - Alarm generation
 - System monitoring
-

1.3 Objectives

- Learn broadcast workflows
 - Understand broadcast devices
 - Visualize signal propagation
 - Simulate real engineering scenarios
 - Demonstrate system design and backend engineering concepts
-

2. Overall System Description

The application consists of a simulation engine that continuously updates the state of every device within the broadcast network.

The simulator follows a tick-based execution model where every simulation cycle updates devices, applies engineering rules, propagates signals, generates alarms, and refreshes the monitoring dashboard.

3. System Architecture

Browser

|

Thymeleaf UI

|

REST + WebSocket

|

Spring Boot

|

Simulation Engine

|

├— Device Manager

├— Connection Manager

├— Rule Engine

├— Signal Propagation Engine

├— Event Engine

├— Alarm Engine

├— Scenario Manager

└— Timeline Manager

|

PostgreSQL

4. Technology Stack

Backend

- Java 21
- Spring Boot
- Spring Web
- Spring WebSocket
- Spring Scheduler
- Spring Data JPA

Frontend

- Thymeleaf
- Bootstrap
- Vanilla JavaScript

Database

- PostgreSQL

Build Tool

- Maven
-

5. Functional Modules

Module 1 — Device Library

Purpose

Provides broadcast devices available for simulation.

Initial Devices

- Camera
- Video Router
- Encoder
- Decoder
- Media Server
- Viewer

Responsibilities

- Create devices
 - Delete devices
 - Display device catalog
-

Module 2 — Topology Builder

Purpose

Allows users to construct a broadcast network.

Features

- Add device
- Connect devices
- Disconnect devices
- Remove devices
- Save topology

- Load topology
-

Module 3 — Device Configuration

Allows editing of engineering properties.

Examples

Camera

- Resolution
- FPS
- Power
- Signal Quality

Encoder

- Codec
- Bitrate
- Resolution
- FPS

Router

- Inputs
- Outputs
- Bandwidth

Media Server

- Storage
 - Capacity
-

Module 4 — Simulation Engine

The core module.

Responsibilities

- Execute simulation ticks
- Update all devices

- Synchronize device states
- Trigger recalculations
- Notify other modules

Simulation Tick

100–500 milliseconds

Workflow

Tick



Update Devices



Apply Rules



Propagate Signals



Generate Alarms



Send WebSocket Updates

Module 5 — Rule Engine

Responsible for engineering calculations.

Example Rules

Increasing FPS



Bandwidth increases



CPU increases



Power increases

↓

Temperature increases

Changing Resolution

↓

Bitrate changes

↓

Bandwidth changes

↓

Network utilization changes

The Rule Engine uses deterministic engineering rules rather than machine learning.

Module 6 — Signal Propagation Engine

Simulates logical video flow.

Example

Camera

↓

Router

↓

Encoder

↓

Viewer

If Camera fails

↓

Router receives NO_SIGNAL

↓

Encoder receives BLACK_FRAME

↓

Viewer loses video

Module 7 — Event Engine

Allows users to inject events.

Initial Events

- Connect
- Disconnect
- Restart
- Power Failure
- Signal Loss
- High CPU
- High Temperature
- Recover Device

Events modify device states and initiate recalculation throughout the simulation.

Module 8 — Alarm Engine

Monitors device health.

Alarm Levels

- INFO
- WARNING
- CRITICAL

Example Conditions

CPU > Threshold

↓

Warning

Temperature > Threshold

↓

Critical

Signal Lost



Critical

Module 9 — Live Dashboard

Displays

- Device Status
- CPU
- Temperature
- Power
- Bandwidth
- Signal Quality
- Alarm Status

Updates occur through WebSockets.

Module 10 — Timeline

Records simulation history.

Example

Camera Connected

Encoder Started

Signal Lost

Temperature Warning

Recovery Complete

Module 11 — Scenario Manager

Allows

- Save topology
- Load topology

- Export JSON
 - Import JSON
 - Reset simulation
-

6. Device Model

Every device follows a common structure.

Common Properties

- Device ID
 - Name
 - Type
 - Status
 - Position
 - Power
 - Temperature
 - CPU Usage
 - Memory Usage
-

Common States

- ONLINE
 - WARNING
 - OFFLINE
 - FAILED
 - RECOVERING
-

Common Operations

- Update
- Recalculate
- Apply Event

- Receive Signal
 - Send Signal
-

7. Device Specifications

Camera

Properties

- Resolution
- FPS
- Signal Quality
- Temperature
- Power
- Status

Outputs

Video Signal

Events

- Disconnect
 - Restart
 - Signal Loss
-

Video Router

Properties

- Inputs
- Outputs
- Active Routes
- CPU
- Temperature

Responsibilities

- Route signals

- Detect disconnected inputs
-

Encoder

Properties

- Codec
- Resolution
- Bitrate
- FPS
- CPU
- Temperature
- Power

Events

- High CPU
 - Overheating
 - Restart
-

Decoder

Properties

- Codec
 - Bitrate
 - CPU
 - Status
-

Media Server

Properties

- Storage Capacity
- Disk Usage
- Current Asset

- CPU

Viewer

Properties

- Resolution
- FPS
- Buffering
- Playback Status

8. Device Profile System

Each device type supports predefined profiles.

Example

Camera

- Generic HD Camera
- Generic 4K Camera

Encoder

- H.264 Encoder
- H.265 Encoder

Profiles define default engineering values.

9. Rule Relationships

Example

FPS ↑

↓

Bandwidth ↑

↓

CPU ↑

↓

Power ↑



Temperature ↑

Another Example

Resolution ↑



Bitrate ↑



Router Bandwidth ↑



Network Load ↑



Viewer Buffering

10. Simulation Flow

Start Simulation



Simulation Tick



Update Device



Apply Engineering Rules



Signal Propagation



Alarm Detection



Timeline Logging



Dashboard Update

11. Database Design

Tables

devices

device_profiles

connections

events

alarms

timeline_logs

saved_scenarios

simulation_settings

12. REST APIs

Examples

Device

- Create Device
- Delete Device
- Update Device
- Get Device

Topology

- Connect Devices
- Disconnect Devices

Simulation

- Start

- Pause
- Resume
- Reset

Scenario

- Save
- Load

Events

- Trigger Event
-

13. WebSocket Updates

The server continuously pushes

- Device metrics
- Alarm updates
- Timeline updates
- Signal changes
- Simulation state

No page refresh is required.

14. Non-Functional Requirements

Performance

- Support 100+ simulated devices
- Dashboard update latency < 1 second
- Stable simulation tick execution

Reliability

- Simulation continues after individual device failures

Maintainability

- Modular architecture
- Easily extendable device types

Portability

- Cross-platform
 - Runs wherever Java is supported
-

15. Assumptions

- Logical simulation only
 - No real video encoding
 - No hardware integration
 - No authentication
 - Single-user environment
-

16. Constraints

- One Spring Boot application
 - Tick-based simulation
 - Rule-based calculations
 - PostgreSQL persistence
-

17. Future Enhancements

- Graphics Engine
- Replay Server
- Audio Mixer
- CDN
- Satellite Uplink
- Multiviewer
- SMPTE ST 2110-inspired stream simulation
- UDP/TCP simulated device communication
- Plugin system for new device types
- AI-based predictive maintenance

- Multi-user collaborative simulations
-

18. Expected Learning Outcomes

By completing BroadcastSim, the developer should gain practical experience in:

- Broadcast infrastructure concepts
 - Simulation engine design
 - Event-driven architecture
 - Real-time systems
 - Spring Boot backend development
 - WebSocket communication
 - Engineering rule modeling
 - System architecture
 - Software design patterns
 - PostgreSQL database design
-

19. Project Vision

BroadcastSim is designed as a Digital Twin of a Broadcast Network.

Rather than acting as a media player or streaming application, it models how professional broadcast devices interact, respond to engineering changes, and propagate faults through a television broadcast chain.

The long-term vision is to evolve BroadcastSim into an extensible educational platform where new broadcast devices, protocols, and engineering rules can be added without modifying the core simulation engine.